

Task 4: Mini Project — Smart Industrial Automation Grid

Robotics & Automation Internship — Progree

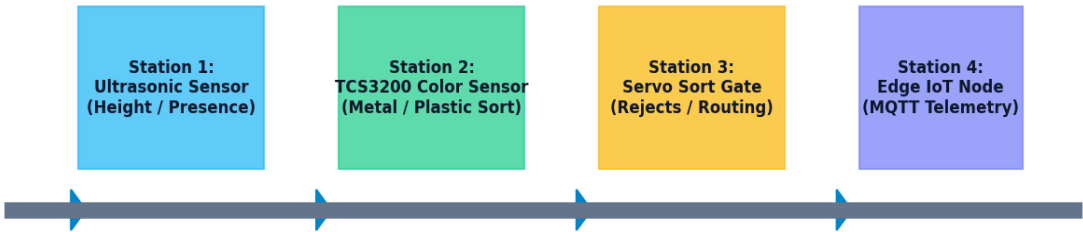
1. Executive Summary & Grid Architecture

This mini-project presents the end-to-end design and control architecture of an automated Industrial Smart Conveyor Sorting Grid. The system integrates ultrasonic height sensing, inductive metal proximity detection, and a high-speed motorized servo sorting diverter gate, coupled with an edge telemetry logging interface.

- Controller: Programmable Logic Controller (PLC) / Microcontroller (Arduino Uno/Mega)
- Sensors: HC-SR04 Ultrasonic Sensor, Inductive Proximity Sensor, TCS3200 Color Sensor
- Actuators: 12V High-Torque DC Conveyor Motor (PWM modulated), MG996R Sortation Servo Gate
- Edge Telemetry: JSON serial packet streaming to Master Station Logger

2. Production Line Assembly Workflow

Smart Industrial Conveyor Automation & Telemetry Grid Architecture



- Step 1 (Ingestion): Raw components enter on the motorized conveyor belt running at nominal 140 PWM speed.
- Step 2 (Dimension Check): Ultrasonic sensor measures distance to determine component presence and height.
- Step 3 (Material/Color Classification): Proximity sensor tests for ferrous metal vs. plastic packaging.
- Step 4 (Automated Sortation): Servo gate actuates (60° deflection) to route metal items into Bin A while allowing plastic items to proceed to Line B.
- Step 5 (Failsafe Jam Detection): If distance < 2.0 cm or sensor blocked for > 2 seconds, conveyor automatically halts (PWM=0) and triggers emergency red alert beacon.

3. Embedded Control & Failsafe Code

Complete control sequencing logic implemented with failsafes:

```

// Progree Robotics & Automation - Task 4
// Smart Conveyor PLC Logic & Edge Telemetry Controller

#include <Servo.h>

const int TRIG_PIN = 7;
const int ECHO_PIN = 8;
const int PROXIMITY_PIN = 2;
const int CONVEYOR_MOTOR_PWM = 9;
const int SERVO_GATE_PIN = 10;
const int LED_STATUS_OK = 11;
const int LED_FAILSAFE = 12;

Servo sortGate;

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(PROXIMITY_PIN, INPUT_PULLUP);
  pinMode(CONVEYOR_MOTOR_PWM, OUTPUT);
  pinMode(LED_STATUS_OK, OUTPUT);
  pinMode(LED_FAILSAFE, OUTPUT);

  sortGate.attach(SERVO_GATE_PIN);
  sortGate.write(0);
  Serial.begin(115200);
}

long measureDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 25000);
  return (duration == 0) ? 999 : (duration * 0.034 / 2);
}

void loop() {
  long distance = measureDistance();
  bool objectDetected = (distance > 2 && distance < 15);
  bool isMetal = (digitalRead(PROXIMITY_PIN) == LOW);

  if (objectDetected) {
    digitalWrite(LED_STATUS_OK, HIGH);
    digitalWrite(LED_FAILSAFE, LOW);
    analogWrite(CONVEYOR_MOTOR_PWM, 180);

    if (isMetal) {
      sortGate.write(60);
      Serial.println("{\"part\": \"METAL_PART\", \"action\": \"DIVERTER_A\"}");
    } else {
      sortGate.write(0);
      Serial.println("{\"part\": \"PLASTIC_PART\", \"action\": \"STANDARD_LINE\"}");
    }
    delay(1500);
    sortGate.write(0);
  } else if (distance < 2) {
    analogWrite(CONVEYOR_MOTOR_PWM, 0);
    digitalWrite(LED_STATUS_OK, LOW);
    digitalWrite(LED_FAILSAFE, HIGH);
    Serial.println("{\"part\": \"JAM_ERROR\", \"action\": \"EMERGENCY_STOP\"}");
    delay(1000);
  } else {
    analogWrite(CONVEYOR_MOTOR_PWM, 140);
  }
  delay(100);
}

```

4. Edge Communication & Master Telemetry Logger

Telemetry packets are serialized in real-time JSON format and archived for SCADA/MES analytics:

```
# Progree Robotics & Automation – Task 4
# Edge IoT Telemetry Receiver & CSV Logger

import json
import csv
import datetime

def simulate_telemetry_stream():
    sample_packets = [
        {"timestamp": datetime.datetime.now().isoformat(), "part": "PLASTIC_PART", "distance_cm": 8.4, "action": "STANDARD_LINE", "conveyor_rpm": 120},
        {"timestamp": datetime.datetime.now().isoformat(), "part": "METAL_PART", "distance_cm": 6.2, "action": "DIVER_A", "conveyor_rpm": 120},
        {"timestamp": datetime.datetime.now().isoformat(), "part": "JAM_ERROR", "distance_cm": 1.2, "action": "EMERGENCY_STOP", "conveyor_rpm": 0}
    ]

    csv_file = "telemetry_production_log.csv"
    with open(csv_file, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["timestamp", "part", "distance_cm", "action", "conveyor_rpm"])
        writer.writeheader()
        for p in sample_packets:
            writer.writerow(p)

    print(f"Logged {len(sample_packets)} telemetry packets into {csv_file}")

if __name__ == "__main__":
    simulate_telemetry_stream()
```